

Lance: 从多模数据湖到 Agent 湖

面向 AI 数据底座的格式、能力与实践

演讲人：马进

AiCon
全球人工智能开发与应用大会

目录

01 Lance: 多模态数据湖格式

02 核心特性与选型对比

03 多模态数据湖实践

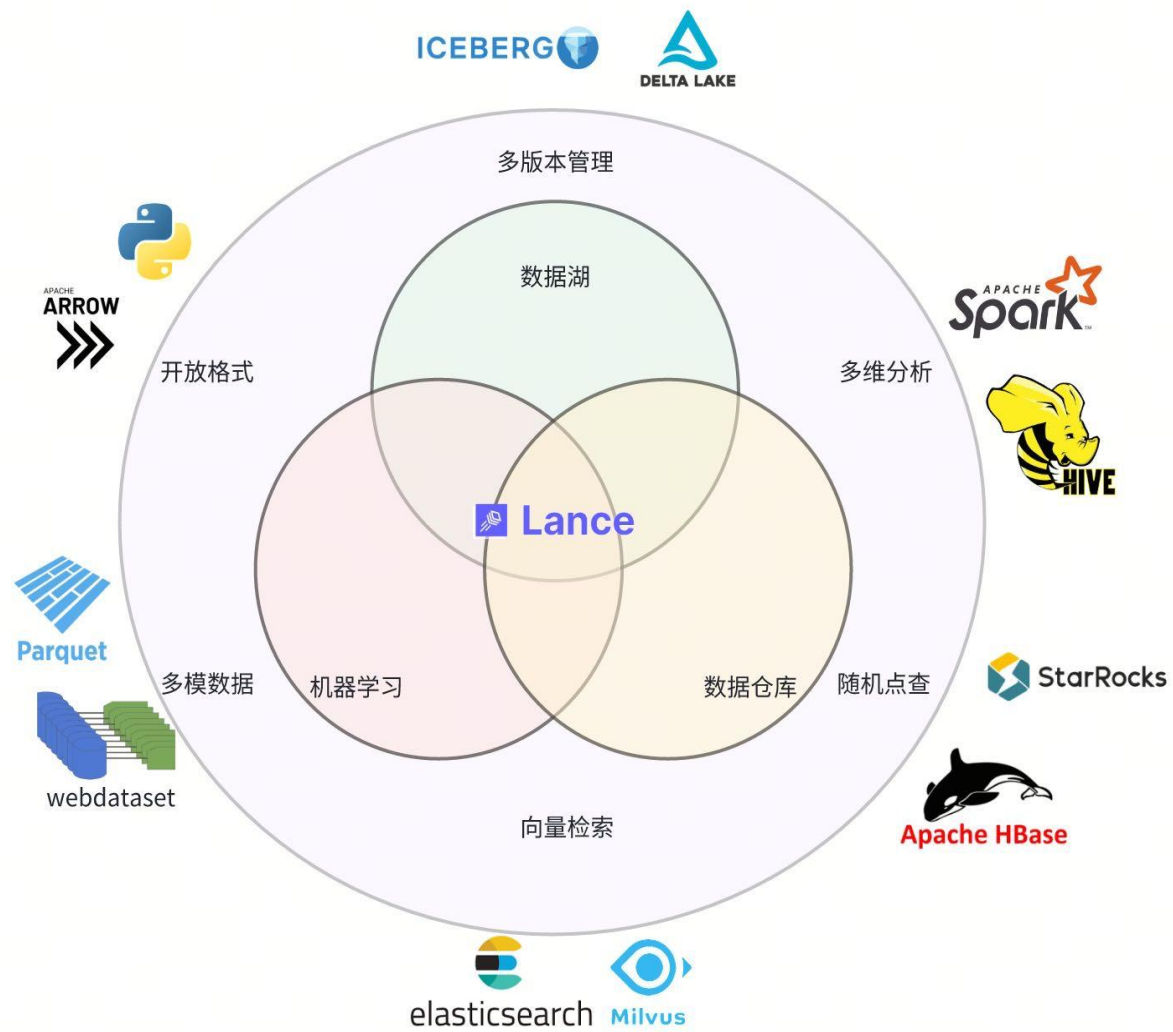
04 基于 Lance 的记忆湖

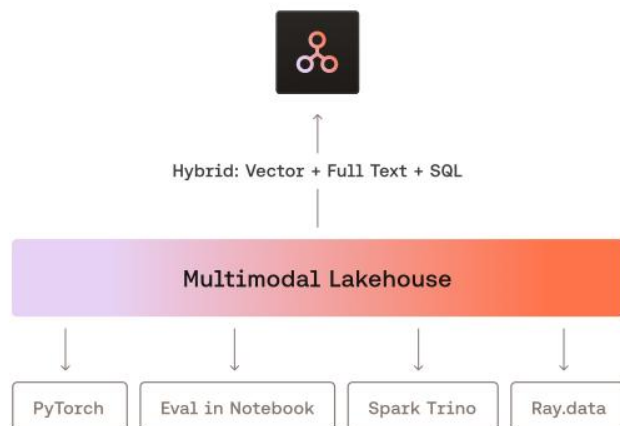
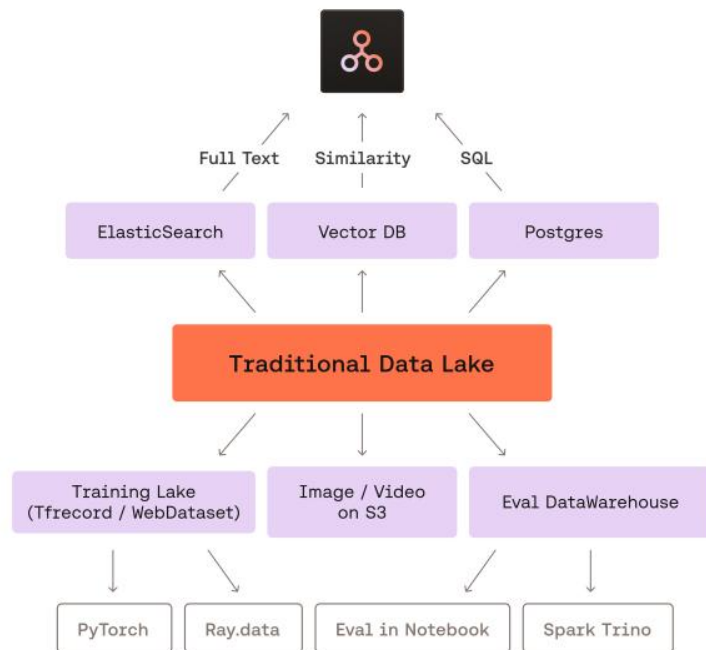
05 总结

01

什么是 Lance

Lance 解决的问题、定位、format stack 与发展历程





传统方案

全文、向量、SQL、训练和评测分散在多套系统里

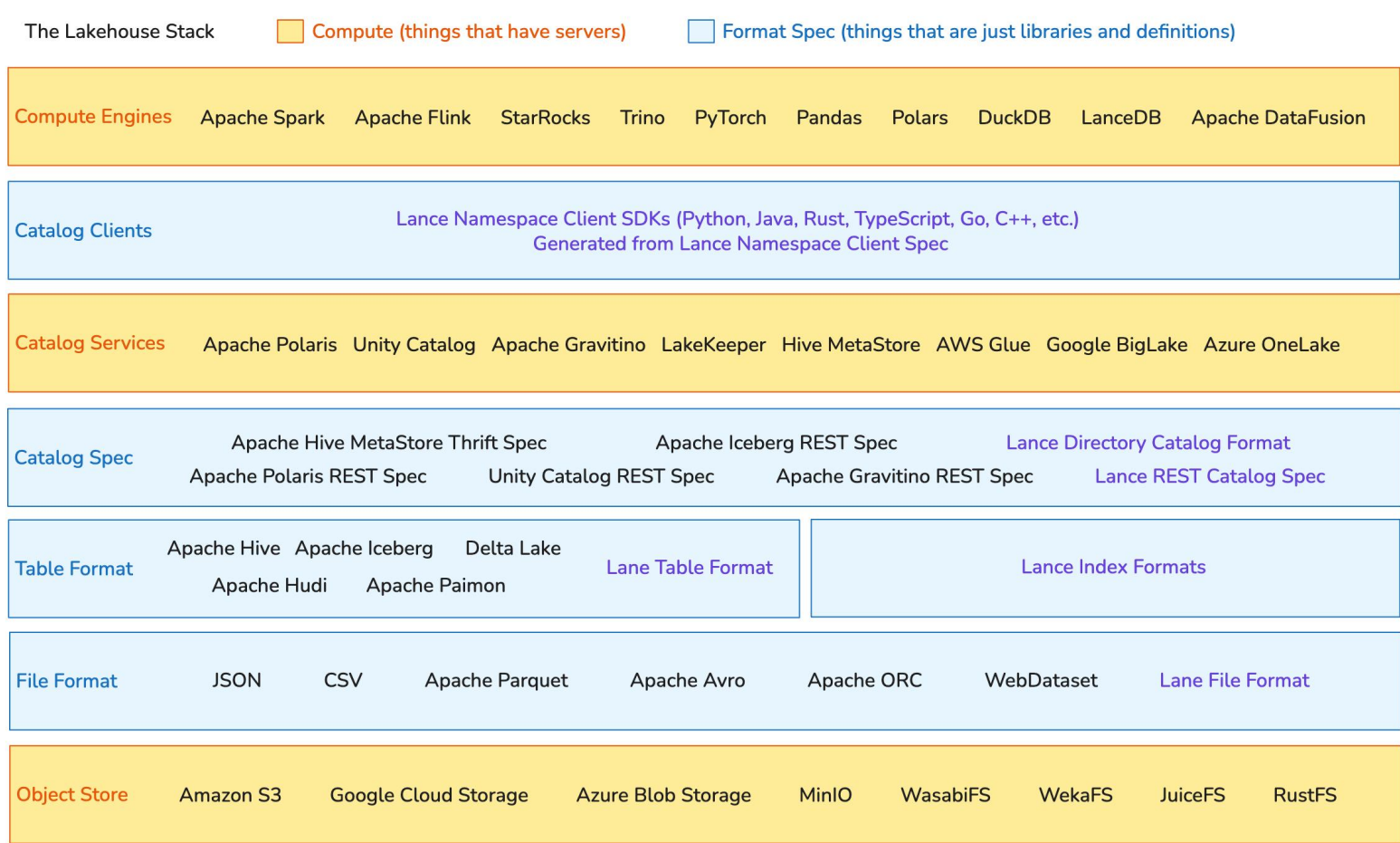
统一查询

Lakehouse 把 Vector + Full Text + SQL 收敛为混合查询能力

统一访问

PyTorch、Spark/Trino、Ray 面向统一数据底座。

01 Lance 是一组互操作规格组成的 lakehouse format stack



File Format

- Column pages / offsets / footer
- 服务随机访问

Table Format

- Fragments / versions / ACID
- 服务表级治理

Index Formats

- Vector / full-text / scalar indices
- 服务混合检索

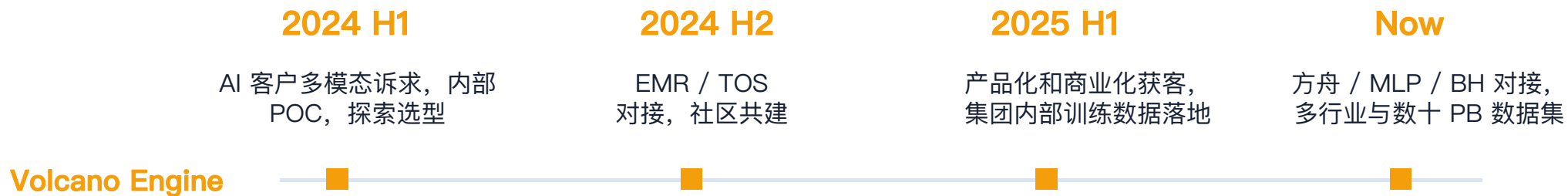
Catalog & Namespace

- Table discovery / coordination
- 服务跨引擎协同

Lance 的演进：开源格式成熟与火山生产实践并行推进



社区格式能力持续成熟，火山生产场景同步验证，让 Lance 从开源技术选型走向 AI 数据湖底座



02

Lance 核心特性，和同类选型对比

随机访问、大宽列、表格式与 Format 2.2

大宽列

多模态数据单值比较大，存储结构需要高效存储图像、音频、视频、点云等大列

结构变更

模型打标和特征工程会不断新增列或更新列，需要低成本 schema evolution

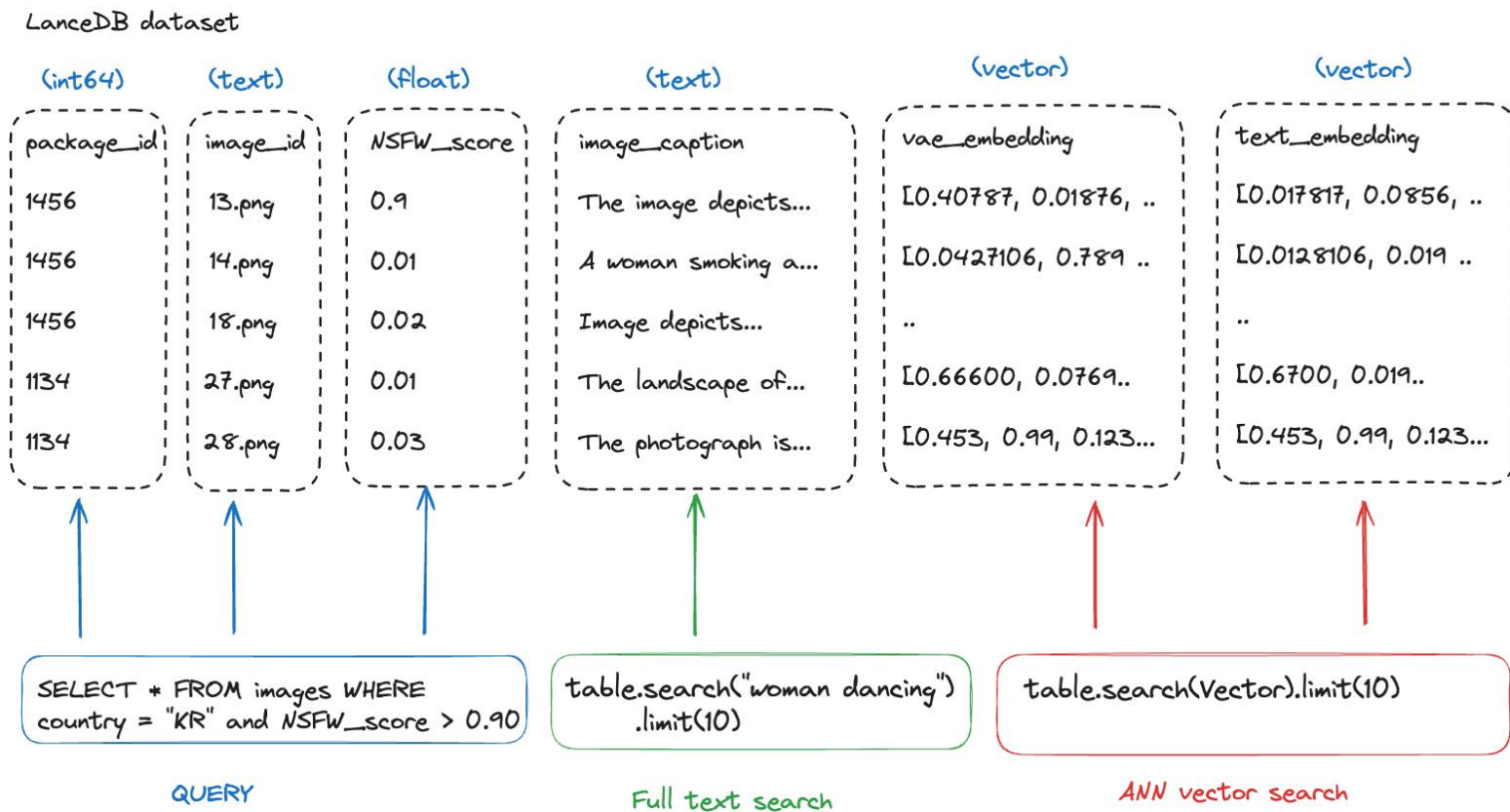
混合存储

结构化、JSON、非结构化对象和向量要放在同一张可查询的数据表里

随机点查

训练 shuffle、样本回放、检索精排都需要稳定的 RowID 随机访问路径

这些诉求共同指向：AI-native 的湖格式能力，而不是单点向量检索能力。



多模存储

结构化、半结构化 JSON、非结构化对象、向量同表协同

随机访问

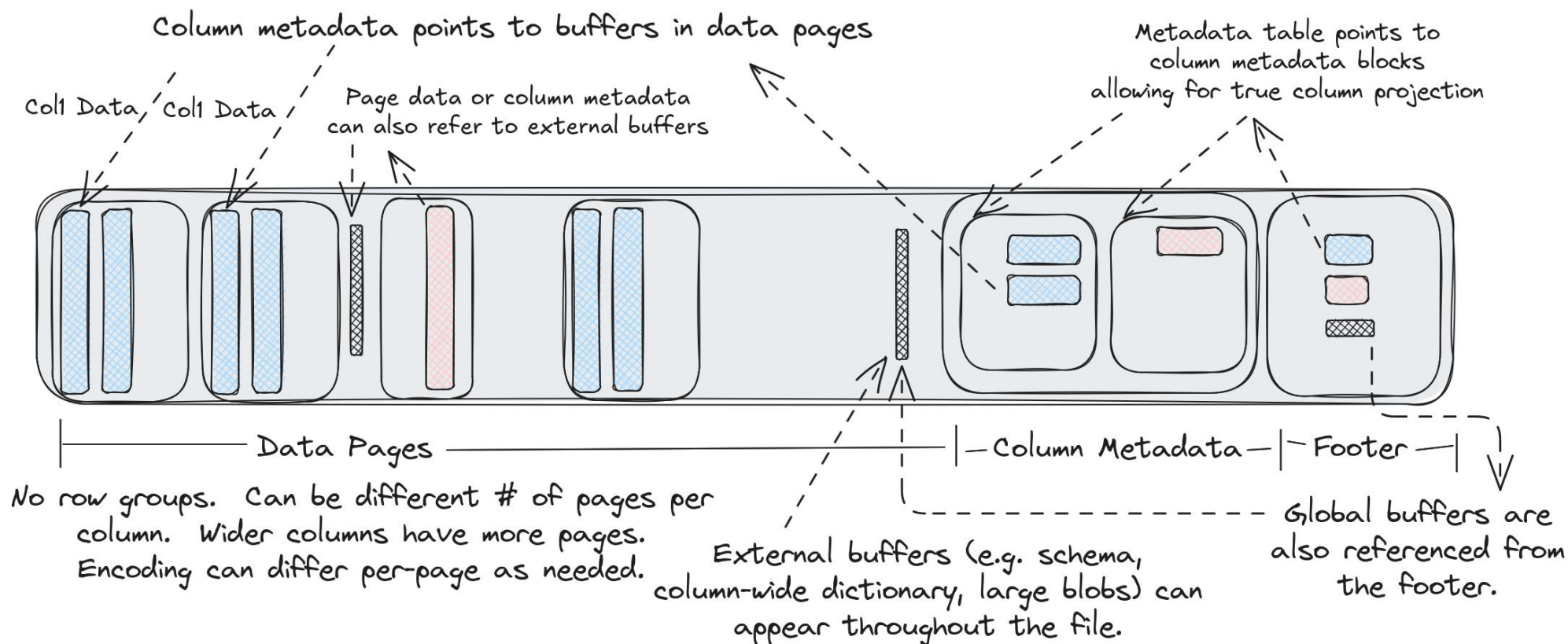
RowID 点查 + 自适应编码，降低训练 shuffle 和检索成本

零成本加列

新增标签或特征时，无需重写整个 Fragment

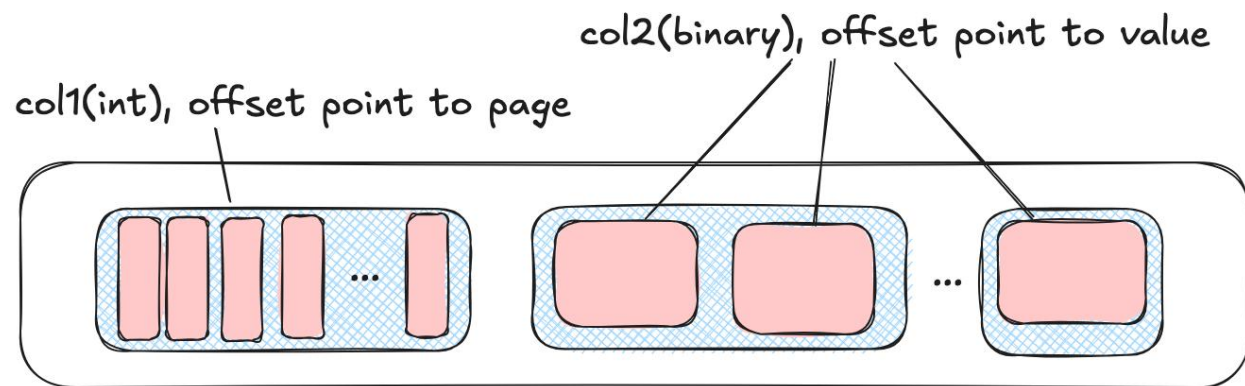
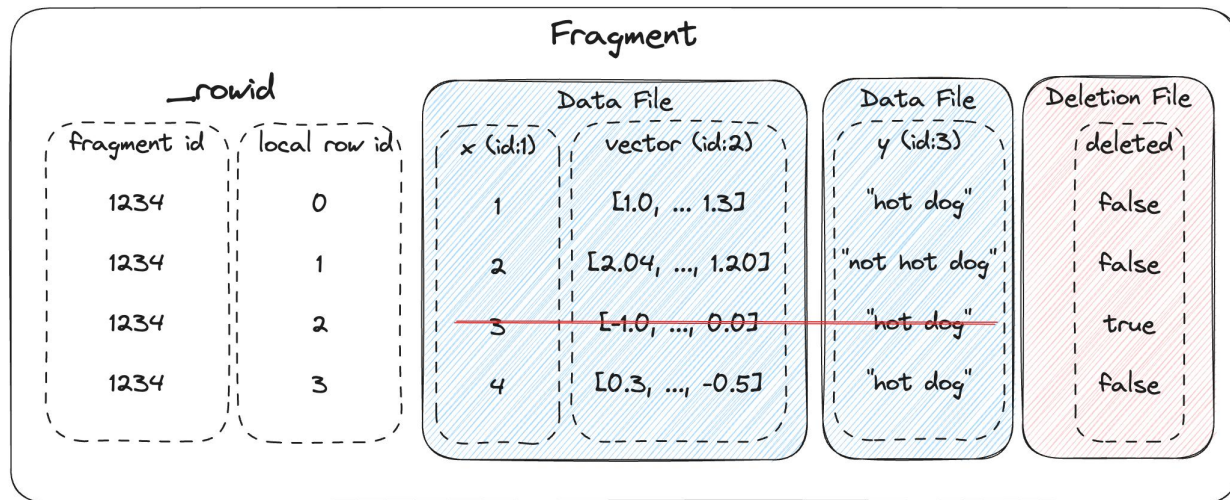
多种索引

Scalar / FTS / Vector 三类二级索引服务混合检索



设计重点: Data / Metadata / Footer 解耦; 移除 Row Group; Page + Column 让目标行定位更直接

随机访问从数十次 IOP 压到 1-2 IOP



1-2

IOP 随机读

Format 2.2 / Fast

100x

随机读取

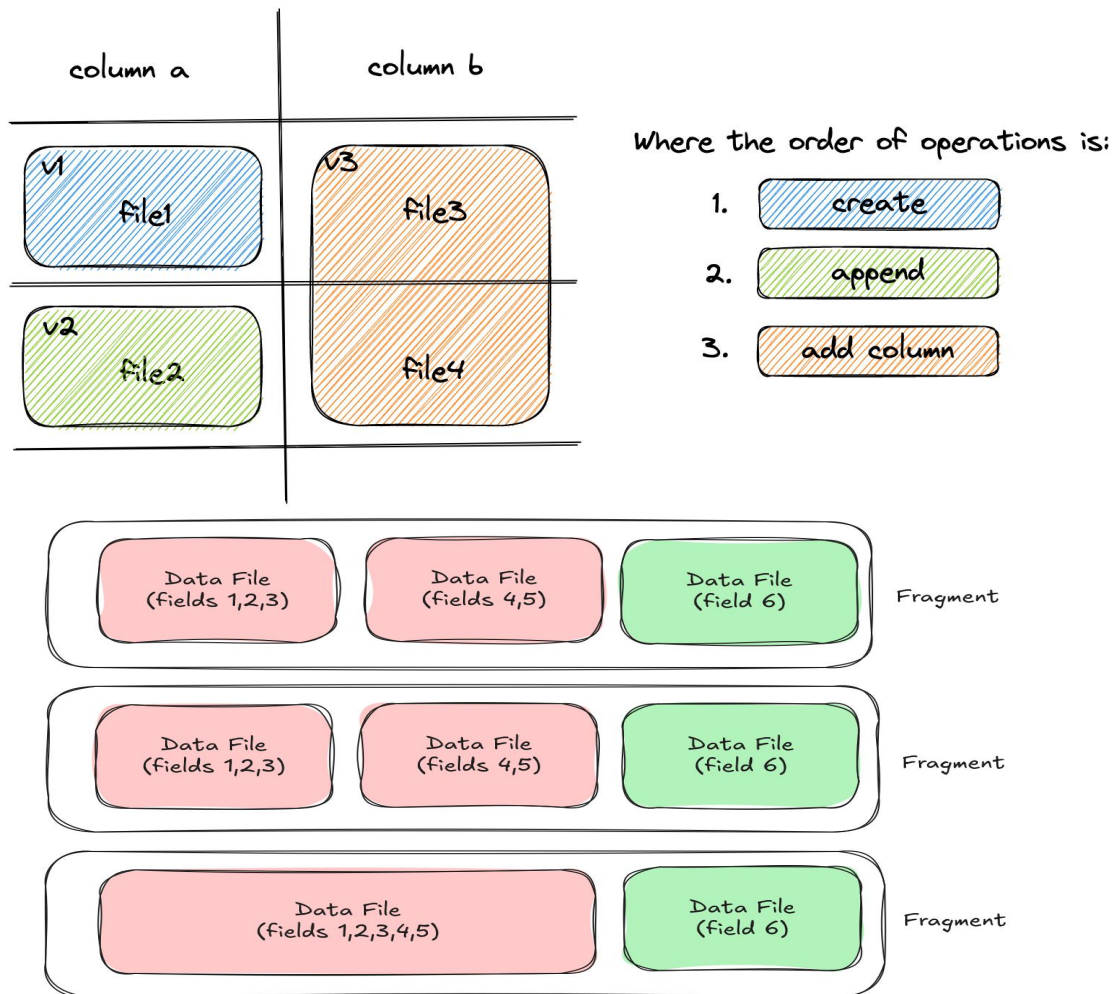
vs Parquet

Blob V2

Lazy Loading

大对象独立通道

02 Table Format 通过 Fragment / DataFile 支持零成本加列和并行优化



Append row

新增行创建新的 DataFile

Append column

不重写 Fragment, 只增加新 DataFile

并行优化

Table 层 Fragment 平衡 IO 与并发

02 Table Format Layout

```
{dataset_root}/
  data/
    *.lance          -- Data files containing column data
  _versions/
    *.manifest        -- Manifest files (one per version)
    latest_version_hint.json -- Optional hint of the latest version (see below)
  _transactions/
    *.txn             -- Transaction files for commit coordination
  _deletions/
    *.arrow           -- Deletion vector files (arrow format)
    *.bin             -- Deletion vector files (bitmap format)
  _indices/
    {UUID}/
      ...             -- Index content (different for each index type)
  _refs/
    tags/
      *.json          -- Tag metadata
    branches/
      *.json          -- Branch metadata
  tree/
    {branch_name}/
      ...             -- Branch dataset
```

Lance 的差异不是替代所有格式，而是把 AI 随机访问和多模态对象变成一等能力

维度	Parquet	Iceberg / Delta	Lance
定位	通用列式文件	表格式/治理层	AI 原生表格式 + 文件格式
随机点查	页 / Row Group 路径长	依赖底层文件格式	1-2 IOP 随机读取路径
大对象	内联或外部管理成本高	不直接定义对象布局	Blob V2 外部通道 + 引用追踪
Schema 演进	宽表更新成本高	表级演进强	Fragment / Column 独立演进
索引	通常依赖外部系统	表级统计/元数据	Scalar / FTS / Vector

03

Lance 多模态数据湖实践

火山引擎 Lance 数据湖架构、生产实践与社区贡献

Slide 17

Table Format 优化

- Branch: 类 Git 分支能力
- Transaction Commit Message
- CDC: Change Data Capture
- 增加时空类型

Performance 提升

- Take API Optimize
- 小数据集点查场景性能瓶颈
- FragmentSession API 优化点查

Index 能力增强

- 分布式创建 FTS Index
- 分布式创建 Vector Index
- JSON 类型支持全文检索
- 无全文索引数据上的 match 检索

计算引擎生态建设

- JavaSDK / Spark Connector
- PyLance / Ray Connector
- PyLance / Daft Connector
- Lance Namespace 共建

SQL 增强

- Lance SDK 新增 SQL API
- UDF: contains_tokens 下推
- UDTF: FTS 检索

LAS 商业化 Feature

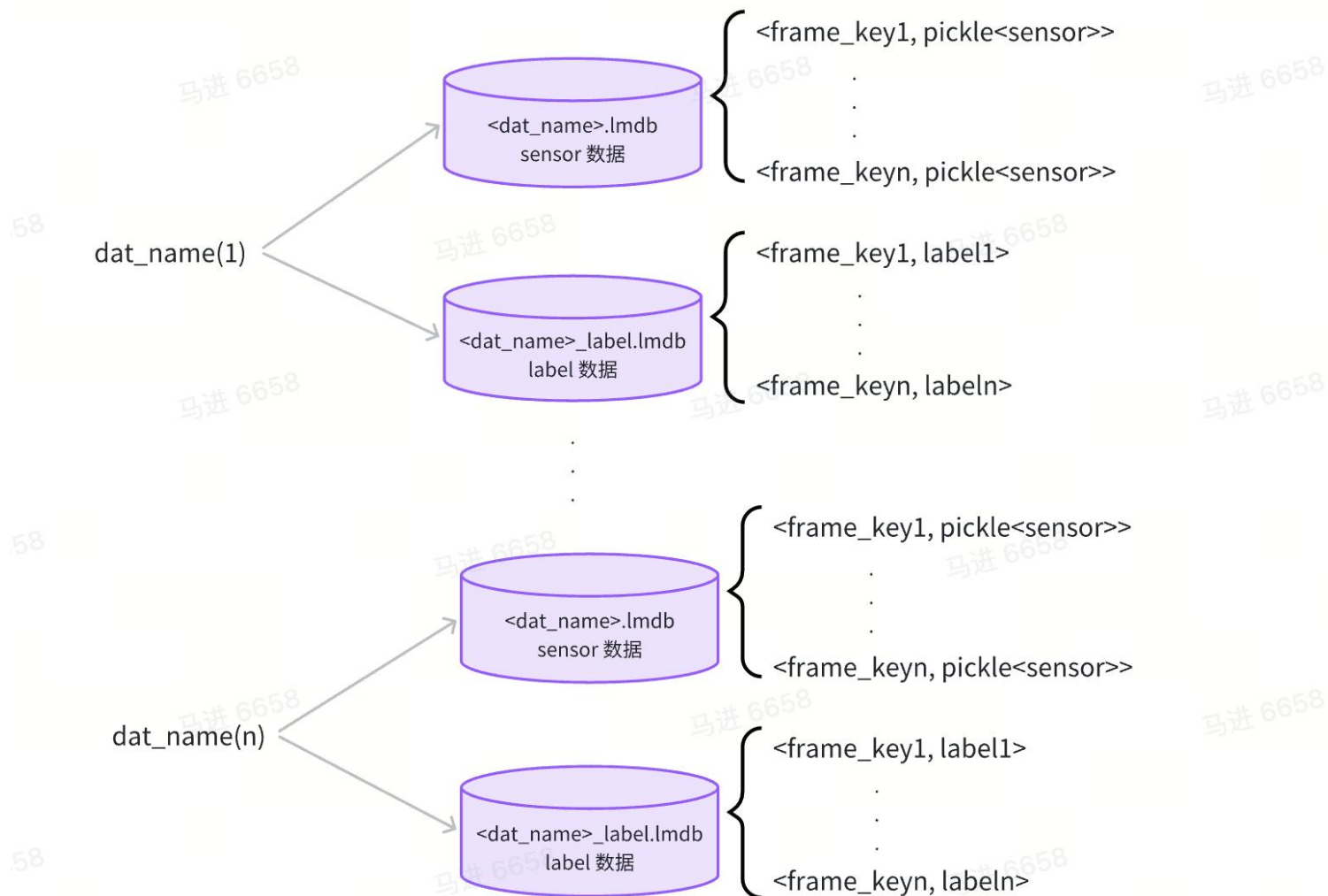
- JXL 压缩: JPEG 格式进一步压缩
- Catalog: 对接 LAS Catalog
- 数据集: 基于 Lance 实现
- 异构存储: vePFS 数据汰换

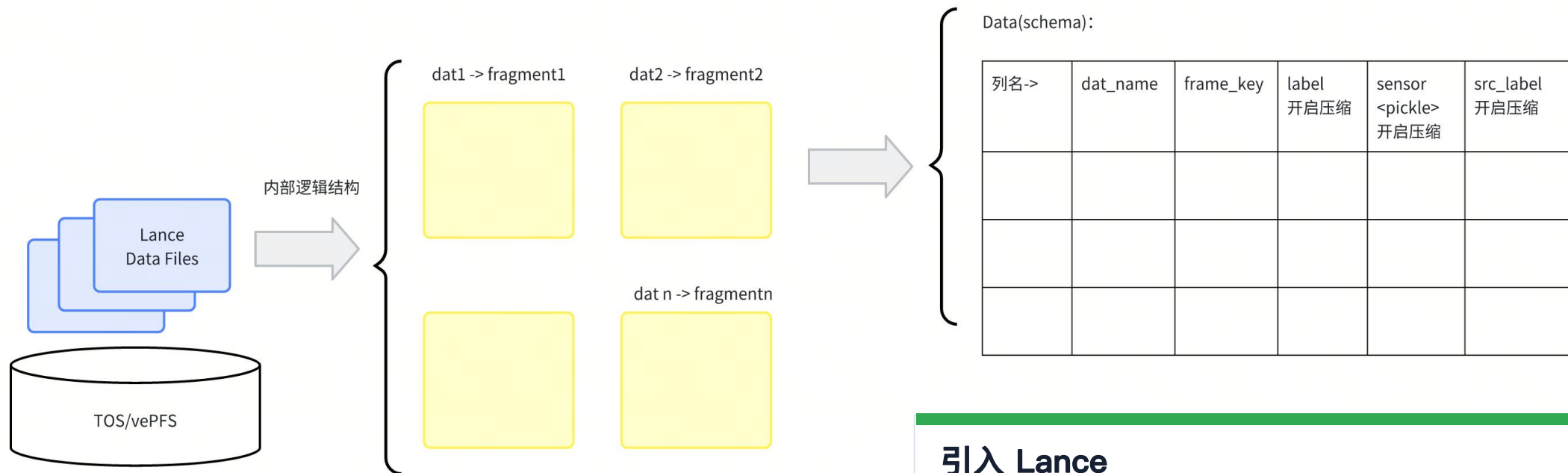
KV 存储多模数据

- 每行数据一帧，一帧几百列
- 包含摄像头（图片）、激光雷达和传感器上报的数据
- 一帧数据的大小约20MB

存在的问题

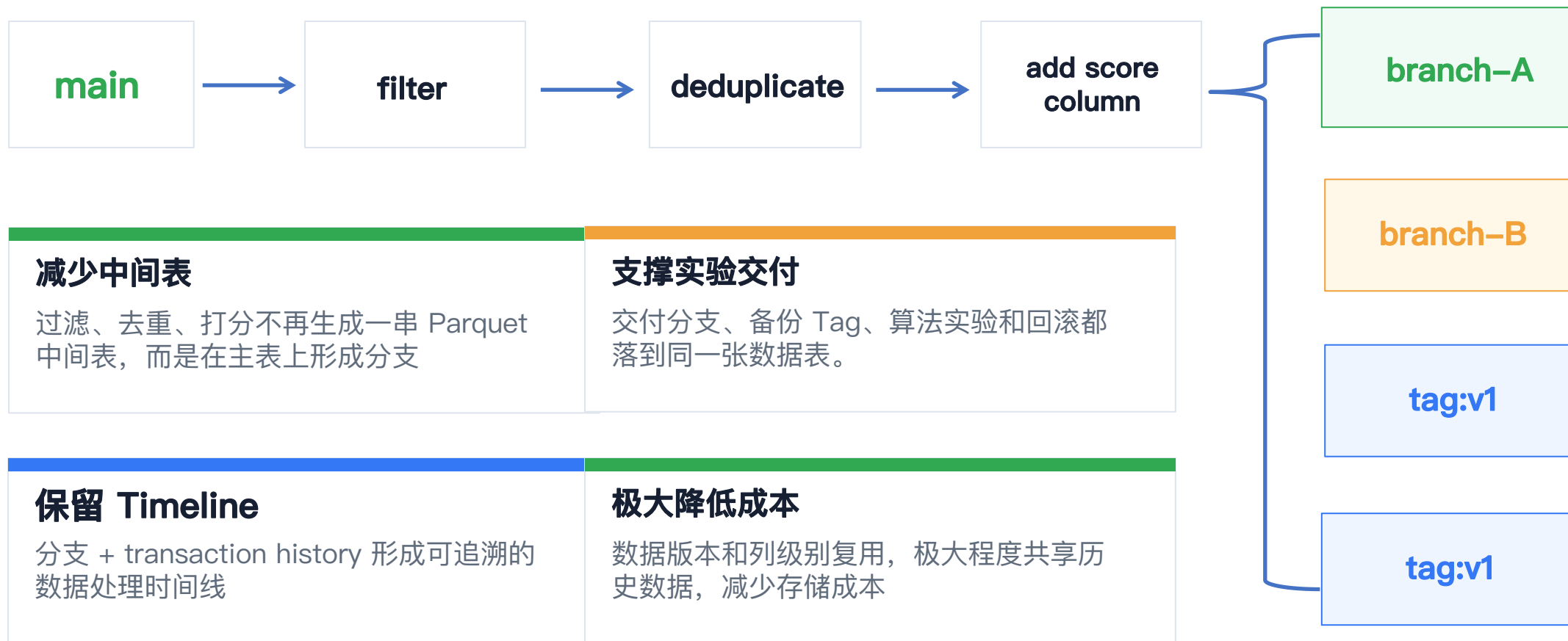
- No schema, 难以治理
- 版本需要不断拷贝
- 压缩效果比较差，占空间
- 并发，扩展能力受限





引入 Lance

- 30% 压缩率，为客户节省了巨额的存储和带宽成本
- 应对了当年 10 倍数据体量的增长





极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO

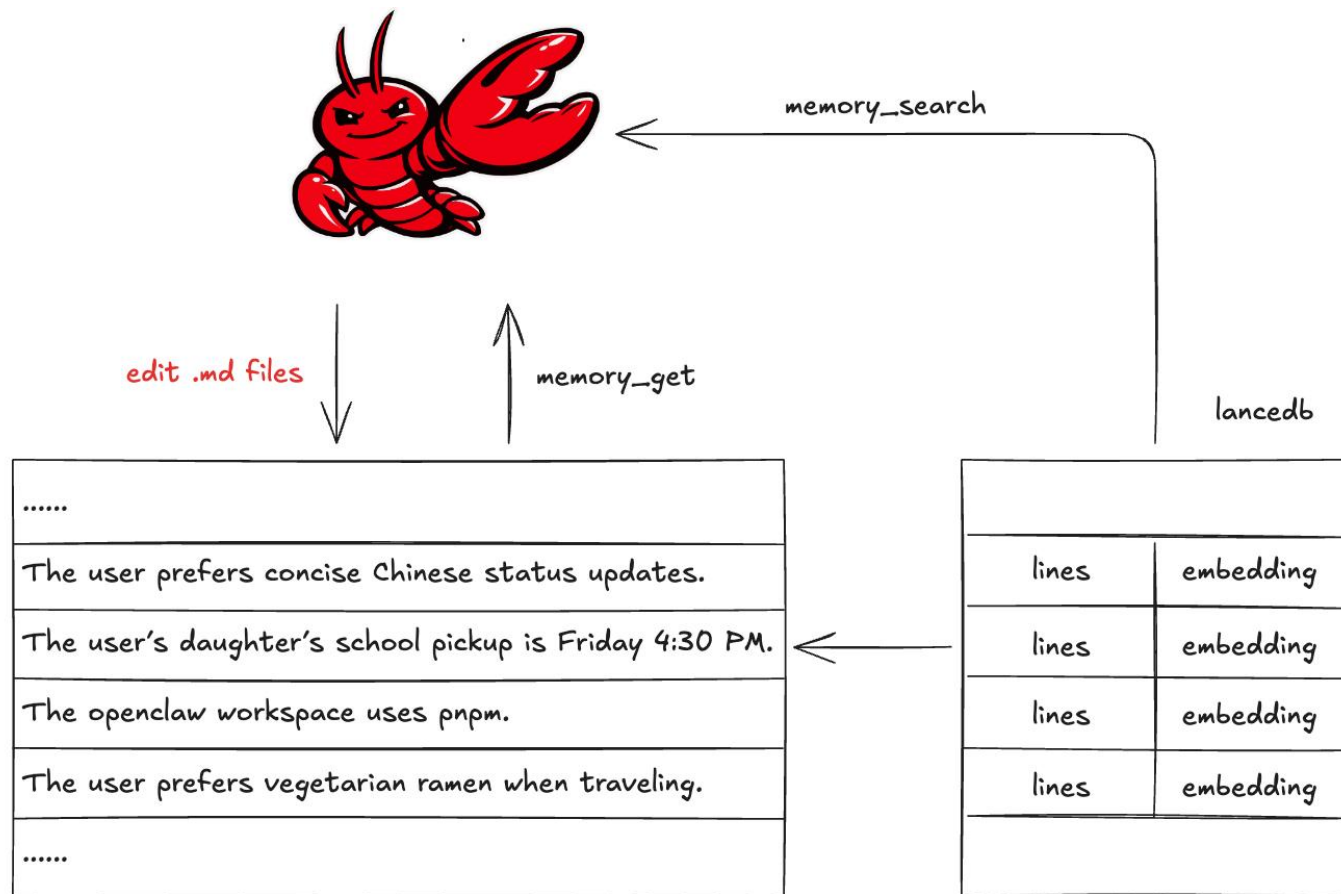


04

基于 Lance 的记忆湖

为什么 Lance / LanceDB 更适配 Agentic workloads

Slide 23



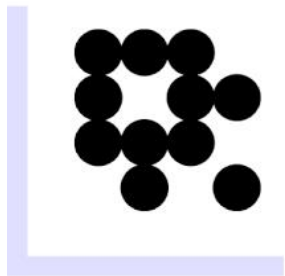
mem0



Anything LLM



OPENCLAW



memory-lancedb-ultra

- Tag 实现毫秒级备份
- Branch 实现团队记忆
- 混合检索 + 中文分词增强
- 每日记忆 + Dreaming 整理



memory-core-lance

- Agentic 记忆 CRUD
- 记忆捕获增强 30%
- 基于语义切分 Chunk, 记忆召回准确率提升 20%
- Session 日志召回



企业场景数据资产

memory-lancedb-ultra

- 更低的响应延迟
- 算子 -> 记忆的 Pipeline
- 企业沉淀知识库 + 知识库演进
- 记忆整理相对困难，成本更高



个人文档，偏好沉淀

memory-core-lance

- 模型维护 Markdown 更强
- 模型天然可以更好地整理记忆
- 社区自带的功能更加丰富
- 会带来更多的 LLM 轮次，会带来更多的响应时间

有没有为这些事情而烦恼

- 不小心归档会话，代码没了
- 长会话之间想要互相引用
- 快速注入历史会话的上下文
- 将上下文整理为可问答知识

Memory 与 Context

- Memory 是 Context 中的高质量和个性化内容提取
- Memory 是自进化的基础
- Context 才是 Agent 核心资产
- Context lake -> 多模态数据湖

上下文，正在形成 Agent 时代的烟囱效应

我用 `using-git-worktrees` 这个流程来把“今天变更的 worktree”定位清楚；你说得对，我刚才对错对象了。现在先重新核对今天的归档会话、它绑定的 cwd、以及 Git 里还有没有可恢复的对象。

🔍 已搜索代码和已列出文件执行了 2 条命令

我看到今天被恢复出来的会话里确实有一个旧日期文件但 mtime 是今天，cwd 指向 `/home/majin.nathan/.codex/worktrees/3171/muninn`；这个目录现在已经不存在，也不在 `git worktree list` 里。接下来我会把今天这些会话逐个按 cwd 和 `fileChange` 过一遍，避免再被文件名日期误导。

🔄 已列出对话线程·已读取对话线程 4 次

对，今天有实质代码变更的是 `019ed12d-fa36-7af2-b502-d7c3639ce117` 这个会话，绑定的 cwd 是 `/home/majin.nathan/.codex/worktrees/3171/muninn`。我已经确认不是刚才那个 `/tmp/muninn-remove_observation`；现在的问题是 `3171` 的目录本身被删了，我在找 Git/Codex 还能不能把未提交内容捞回来。

我会先做只读恢复检查：`git fsck` 不写 `lost-found`，只列 `dangling/unreachable` 对象，然后用今天改过的关键词去搜对象内容。如果能命中完整文件版本，就比手工从对话摘要重写靠谱得多。

🔍 已运行 3 条命令

有个重要线索：Git 里有 `refs/codex/snapshots/...` 这种 Codex 快照引用。这类引用有可能保存过工作区快照，我现在优先查它们和 `dangling commits`，看看今天那几处文件有没有完整版本。

正在思考

Lance 把多模态对象、向量、全文、SQL 和 Agent 记忆收敛到同一套数据底座

统一访问

同一张表同时承载原始对象、结构化字段、全文、embedding 与索引，减少跨系统搬运

格式即能力

File / Table / Index / Catalog 分层演进，把随机访问、加列和治理变成底层能力

面向 AI 流程

训练、检索、评测、打标和记忆都围绕同一份可版本化数据展开